Kafka HA Roadmap — 4 Weeks

Goal: Design and defend highly available Kafka solutions (SRE/infra lens), with Confluent Cloud + an edge cluster as the real-world target. **Time budget:** ~1–1.5 h on weekdays, ~3 h lab on the weekend. **Rule:** Every concept gets *broken on purpose* in a lab. Never on your production edge node.

Lab setup (do before Day 1)

- A 3-broker KRaft cluster on Docker Compose (latest Apache Kafka 4.x image or Confluent Platform). It needs 3 brokers, because you can't observe ISR behaviour on 1.
- Tools: `kafka-topics`, `kafka-console-producer/consumer`, `kafka-producer-perf-test`, `kafka-consumer-groups`.
- A small Python or Java producer and consumer where you control the config (confluent-kafka-python is fine).
- Optional but valuable: Toxiproxy, to inject latency and network partitions.
- A single-node container that mirrors your edge setup (used in Week 4).

Week 1: Producer internals & durability

Outcome: Explain exactly what happens to a record from `send()` to acknowledged, and under every failure.

Day	Topic	Task
1	Send path	Metadata fetch, partitioner (sticky vs key hash), RecordAccumulator, sender thread. Draw it from memory.
2	Batching	`batch.size`, `linger.ms`, `compression.type`, `buffer.memory`, `max.block.ms`. What blocks and what throws?
		`acks` (0/1/all), ISR, `min.insync.replicas`,

3	Durability	`replica.lag.time.max.ms` . Why is acks=all with RF=1 the same as acks=1?
4	Retries & timeouts	`retries` , `retry.backoff.ms` , `request.timeout.ms` , `delivery.timeout.ms` . Which one actually decides when a send fails?
5	Idempotence & ordering	Producer ID + sequence numbers, `max.in.flight.requests.per.connection` ≤ 5, why reordering used to happen, and what idempotence does *not* cover.

Weekend lab

1. RF=3, min.insync=2, acks=all. Run perf-test, kill 1 broker, then a second. Record the latency spike, the errors, and the time until writes fail.
2. Repeat with acks=1 and compare what gets lost when the leader dies right after acking.
3. Fill the buffer: stop all brokers while producing and watch `buffer.memory` and `max.block.ms` take effect.

Progress check (explain to me, no notes)

- Walk a record through the producer, naming every buffer and thread.
- Give 3 ways a "successfully sent" record can still be lost.
- Explain why `delivery.timeout.ms` must be ≥ `linger.ms` + `request.timeout.ms` .

Week 2: Consumer internals & delivery semantics (your weakest area)

Outcome: Predict exactly which records get duplicated or skipped in any consumer failure.

Day	Topic	Task
1	Poll loop & fetch	`fetch.min.bytes` , `fetch.max.wait.ms` , `max.poll.records` , `max.partition.fetch.bytes` . What does one `poll()` actually do?
	Group	Group coordinator, heartbeat thread, `session.timeout.ms` vs

2	membership	`max.poll.interval.ms` . Why are there two timeouts?
3	Rebalancing	Eager vs cooperative-sticky, the new broker-driven protocol (KIP-848), and static membership (`group.instance.id`).
4	Offset commits	Auto-commit timing, sync vs async commits, commit-before vs commit-after processing, and `auto.offset.reset` .
5	Delivery semantics	At-most-once, at-least-once, and exactly-once (transactions + `read_committed`). Why EOS stops at Kafka's boundary.

Weekend lab

1. Make a consumer sleep longer than `max.poll.interval.ms` . Log the rebalance, the CommitFailedException and the duplicated records.
2. Run 3 consumers and kill one: compare pause time under eager vs cooperative-sticky assignment.
3. Crash a consumer mid-batch with auto-commit on, then with manual commit after processing. Count the duplicates and losses.
4. Use `kafka-consumer-groups --describe` and `--reset-offsets` (dry run first).

Progress check

- Re-answer the original Q2 in full, step by step.
- Design a consumer for a payment event where duplicates are unacceptable but losses are worse. What goes where, and what sits outside Kafka?
- Explain when static membership helps and when it hurts.

Week 3: Replication, Cluster Linking & DR

Outcome: Quantify RPO/RTO for a multi-cluster setup and write the failover procedure.

Day	Topic	Task
1	Replication internals	Leader/follower, LEO vs high watermark, why consumers only see up to the high watermark, and leader epochs.
2	Failure edges	Unclean leader election, rack awareness, and what Confluent

		Cloud manages for you vs what it doesn't.
3	Cluster Linking	Mirror topics, byte-for-byte offset preservation, source-initiated links (your edge→CC setup), link lag metrics.
4	Offset & ACL sync	`consumer.offset.sync.enable`, sync interval, consumer group filters, and why offset sync lagging data causes reprocessing.
5	DR patterns	Active-passive vs active-active, failover vs promote, failback, and client redirection (DNS/config). How it compares with MirrorMaker 2.

Weekend lab

1. Kill a follower in your 3-broker lab and observe ISR shrink/expand and the high watermark behaviour.
2. If you have a non-prod CC environment: link two clusters, produce continuously, fail over, and measure lost and duplicated records.
3. Write a failover runbook: trigger criteria, commands, verification, and rollback.

Progress check

- Re-answer the original Q3 with numbers for RPO, RTO, duplicates and loss.
- Explain why identical offsets make Cluster Linking failover simpler than MirrorMaker 2.
- Given a link lag of X seconds and an offset sync of Y seconds, what's the worst case?

Week 4: HA design & SRE practice (capstone)

Outcome: A reviewed HA design for your own edge→Confluent Cloud architecture.

Day	Topic	Task
1	Observability	Key metrics: consumer lag, under-replicated/under-min-ISR partitions, request latency, record-error-rate, link lag. Define SLIs and SLOs.
	Edge failure	List every way your edge path fails: node down, disk loss, WAN

2	modes	outage, CC outage, link paused.
3	Edge HA options	Multi-node edge vs single node + durable producer-side buffering vs producing directly to CC. Weigh cost, complexity and RPO.
4	Chaos & capacity	Plan failure injection, partition count and throughput sizing, and retention for a WAN outage (how many hours can the edge hold?).
5	Design doc	Write a 2–3 page design: architecture, failure matrix, RPO/RTO per failure, runbooks and monitoring.

Weekend: final review

- Bring me the design doc. I'll attack it with failure scenarios and you defend it.
- Re-take all 3 original assessment questions cold.

Scoring rubric (per area, per check)

- **1: Recall.** You can name the configs.
- **2: Mechanism.** You can explain what happens internally.
- **3: Prediction.** You can predict the outcome of a failure before testing it.
- **4: Design.** You can choose configs and architecture from stated RPO/RTO requirements.

Target by end of Week 4: level 4 for producer and multi-region, level 3+ for consumer.

Current placement: Producer 2, Consumer 1, Multi-region 1–2.